

9.0 tuplas, conjuntos y diccionarios

Programación I
Ingeniería Electrónica y Telecomunicaciones

Comisión 3
Profesor: Lautaro Linquimán
Universidad Nacional de Río Negro



Repaso express de la clase anterior 29/04

- Repasamos al final tema examen

Tuplas – Tuple



¿Qué son las tuplas?

Una tupla es una colección ordenada de valores inmutables.

Son parecidas a las listas en formato y en sus operaciones, pero tienen la gran diferencia de que una vez creada una tupla y no se puede modificar.

Para definir una lista utilizamos paréntesis y el contenido separado por “,”

```
mi_primera_tupla = ("Lautaro", "Linquimán")
mi_segunda_tupla = (1, 2, 3)

print(mi_primera_tupla)
```

Definamos la tupla y revisemos de qué tipo es con `type(mi_primera_tupla)`

Tuplas ()

La variable de tipo tupla nos permite almacenar diferentes valores una sola variable.

Comparte varias de las operaciones con las listas

Contenido puede ser de tipo string, entero, flotante, otras listas, etc.

EJEMPLOS

```
# Crear una tupla.  
persona = ("Ana", 20) # Una forma de crear una tupla  
persona = tuple(("Ana", 20)) # Otra forma de crear una  
tupla
```

```
# Acceder por índice.  
print(persona[0])  
print(persona[-1])
```

```
# Desempaquetar valores.  
nombre, edad = persona
```

```
print(nombre)  
print(edad)
```

```
# Recorrer con un for  
for x in persona:  
    print(x)
```

```
# INMUTABLE  
persona[0] = "Pablo"
```

¿Cuándo usar tuplas?

Podemos usar tuplas cuando podemos agrupar datos estructurados que no necesitamos mutar, por ejemplo:

- Coordenadas geográficas.
- Fechas
- **Resultado de una función**

```
coordenada = (-41.13, -71.31)
```

```
fecha = (5, 5, 2026)
```

```
def operaciones(a, b):  
    return (a + b, a - b)
```

```
suma, resta = operaciones(5, 10)
```

Definamos la lista y revisemos de qué tipo es con `type(lista_de_super)`

Ejercicio de tuplas

EJ1: Crear una función que reciba un nombre y una edad, y devuelva una tuple con:

- el nombre
- la edad
- un booleano que indique si es mayor de edad

Ejemplo:

```
datos_persona("Lauti", 20)
```

```
("Lauti", 20, True)
```



Conjuntos – Set

¿Qué son los conjuntos?

Un conjunto es una colección de elementos únicos, sin repetidos y sin un orden importante.

Podríamos usar conjuntos para tener:

1. Lista de asistentes únicos.
2. DNI de alumnos.

```
mi_primer_conjunto = {1, 2, 3, 3, 4}
print(mi_primer_conjunto)
print(len(mi_primer_conjunto))
```

Definamos el conjunto y revisemos de qué tipo es con

```
type(mi_primer_conjunto)
```

Conjuntos {}

La variable de tipo conjunto nos permite almacenar diferentes valores sin duplicados.

Contenido puede ser de tipo string, entero, flotante, tupla, pero OJO no puede contener otros tipos mutables como listas, otros conjuntos o diccionarios.

EJEMPLOS

```
# Crear set
```

```
nombres = {"Ana", "Juan", "Pedro"}
```

```
print(nombres)
```

```
# Agregar con add
```

```
nombres.add("Lucía")
```

```
print(nombres)
```

```
# Eliminar con remove
```

```
nombres.remove("Juan")
```

```
print(nombres)
```

```
# Consultar pertenencia con in
```

```
print("Ana" in nombres)
```

```
# Convertir lista a set para eliminar repetidos
```

```
lista_nombres = ["Ana", "Juan", "Ana", "Pedro"]
```

```
nombres_unicos = set(lista_nombres)
```

```
print(nombres_unicos)
```

Conjuntos {}

Se pueden aplicar teoría de conjuntos

EJEMPLOS

```
# Sets: unión, intersección y diferencia
```

```
a = {"Ana", "Juan", "Pedro"}
```

```
b = {"Juan", "Lucía"}
```

```
print("Grupo A:", a)
```

```
print("Grupo B:", b)
```

```
# Unión: junta los elementos de ambos grupos
```

```
print("Unión:", a | b)
```

```
# Intersección: muestra los elementos que tienen en común
```

```
print("Intersección:", a & b)
```

```
# Diferencia: muestra qué elementos están en A pero no en B
```

```
print("Diferencia A - B:", a - b)
```

Ejercicio de conjuntos

EJ2: Dada una lista con nombres repetidos, mostrar:

- los nombres únicos
- la cantidad de nombres distintos

Código base:

```
nombres = ["Ana", "Juan", "Ana", "Pedro", "Juan", "Lucía"]
```



Diccionarios

¿Qué son los diccionarios?

Los diccionarios en Python son una estructura de datos que permite almacenar su contenido en forma de clave valor.

Desde mi perspectiva de los tipos de datos que mayor versatilidad nos dan en el lenguaje.

Nos permite construir de forma simple:

- Ficha de alumno.
- Agenda de contactos.
- Diccionario real: palabra -> definición.

```
mi_primer_diccionario = {  
    "clave_1": "valor_1",  
    "clave_2": "valor_2",  
}
```

Definamos el diccionario y revisemos de qué tipo es con `type(mi_primer_diccionario)`

¿Qué son los diccionarios?

Los diccionarios son muy útiles porque nos permiten estructurar y acceder a nuestros datos con una semántica muy clara (si lo usamos bien).

Comparemos los casos de uso de una tupla vs un diccionario:

```
alumno_tupla = ("Paula", "Pérez", 8)
print(alumno_tupla[2])
```

```
alumno_dict = {
    "nombre": "Paula",
    "apellido": "Pérez",
    "nota": 8
}
print(alumno_dict["nota"])
```

Diccionarios

{key: value}

La variable de tipo diccionario nos permite almacenar diferentes valores siempre y cuando no se duplique la key.

Si se duplica la key, se pisa el valor.

Las keys de nuestro diccionario pueden ser int, float, str o bool. Mientras que el value puede ser prácticamente cualquier cosa.

EJEMPLOS

```
# Crear un diccionario
persona = {
    "nombre": "Martín",
    "edad": 20
}

# Acceder a un valor usando su clave
print(persona["nombre"])

# Modificar un valor existente
persona["edad"] = 21

# Agregar una nueva clave
persona["ciudad"] = "Bariloche"

# Preguntar si existe una clave
print("ciudad" in persona)

# Obteniendo las Keys
keys = persona.keys()

# Obteniendo los valores
values = persona.values()
```


Recorriendo Diccionarios {key: value}

Los diccionarios se pueden recorrer de varias maneras:

- Por clave
- Por clave y valor
- Por valor

EJEMPLOS

```
persona = {  
    "nombre": "Martín",  
    "edad": 21,  
    "ciudad": "Bariloche"  
}
```

```
# Recorrer por claves
```

```
for clave in persona:  
    print(clave, persona[clave])
```

```
# Recorrer por clave y valor
```

```
for clave, valor in persona.items():  
    print(clave, valor)
```

```
# Recorrer solo las claves
```

```
for clave in persona.keys():  
    print(clave)
```

```
# Recorrer solo los valores
```

```
for valor in persona.values():  
    print(valor)
```

Ejercicio con diccionarios

EJ3: Crear un diccionario que represente un producto con:

- nombre
- precio
- stock

Luego:

1. Mostrar el producto completo.
2. Aumentar el precio en un 10%.
3. Restar una unidad al stock.
4. Mostrar un mensaje final con este formato:

Producto: Nombre - Precio actualizado: Precio - Stock restante: Stock

EJ4: Crear un diccionario que represente una cuenta de usuario con:

- usuario
- email
- activo

Luego:

1. Mostrar el email del usuario.
2. Cambiar el estado activo a False.
3. Agregar una nueva clave llamada ultimo_login.
4. Mostrar el diccionario completo.

Estructuras combinadas

Listas, tuplas, conjuntos y diccionarios, todo en un solo lugar

Las estructuras se pueden combinar

Es normal durante el desarrollo de programas que una estructura por sí sola no sea completamente funcional.

En estos casos se suele recurrir a combinar estructuras para simplificar el programa.

Estas estructuras se pueden recorrer y operar de forma individual.

```
alumno = {  
    # str  
    "nombre": "Ana",  
    # list  
    "notas": [8, 6, 9],  
    # tuple  
    "ubicacion": ("Bariloche", "Río Negro"),  
    # set (conjunto)  
    "materias": {"Programación", "Matemática"}  
}
```

```
nota_total = 0  
for nota in alumno["notas"]:  
    nota_total += nota
```

Listas de diccionario

Una lista de diccionarios es una estructura muy útil en Python porque permite guardar varios elementos con información organizada.

Cada diccionario representa un elemento, y la lista agrupa todos esos elementos.

En el ejemplo vemos una lista de diccionarios con información de los alumnos.

Podemos manipularla como a una simple lista.

PERO CUIDADO: evita introducir en estas listas elementos que no compartan estructura dado que podría inducir fallos en nuestro programa si no lo hacemos bien.



```
alumnos = [  
    {"nombre": "Paula", "nota": 8},  
    {"nombre": "Juan", "nota": 3},  
    {"nombre": "Pedro", "nota": 6}  
]  
  
alumnos.append(  
    {"nombre": "Jhon", "nota": 2}  
)  
  
for alumno in alumnos:  
    if alumno["nota"] >= 4:  
        print(alumno["nombre"], "aprobó")  
    else:  
        print(alumno["nombre"], "desaprobó")
```

Otros ejemplos

```
# Un diccionario con un diccionario
alumno = {
    "nombre": "paula",
    "contacto": {
        "email": "paula@mail.com",
        "telefono": "2944000000"
    }
}
```

```
curso = {
    "nombre": "Programación 1",
    "comision": 3,
    "alumnos": ["Paula", "Juan", "Pedro"]
}
```

Ejercicio integrador

EJ5: Tenemos una lista de alumnos. Cada alumno está representado con un diccionario.

Cada alumno tiene:

- nombre
- notas
- materias

Hay que hacer lo siguiente:

- Mostrar todos los nombres.
- Mostrar quiénes aprobaron.
- Mostrar quiénes cursan "Matemática".
- Agregar "Laboratorio" a las materias de Joaquin.

```
alumnos = [  
    {  
        "nombre": "Joaquin",  
        "nota": 8,  
        "materias": {"Programación", "Matemática"}  
    },  
    {  
        "nombre": "Juan",  
        "nota": 3,  
        "materias": {"Programación"}  
    },  
    {  
        "nombre": "Lucía",  
        "nota": 9,  
        "materias": {"Programación", "Inglés"}  
    }  
]
```

PDFS y TAREAS



Todo el material en <https://github.com/Ymil/unrn-programacion-1/>

FIN

Volvemos el miércoles a las 17